

Использование null-выражений в Kotlin

Прежде чем начать

Этот коделаб расскажет вам о null и важности безопасности null. Нуллифицируемость - это понятие, широко распространенное во многих языках программирования. Оно означает способность переменных не иметь значения. В Kotlin нуллифицируемость специально обрабатывается для достижения безопасности null.

Необходимые условия

- Знание основ программирования на Kotlin, включая переменные, доступ к методам и свойствам из переменной, а также функции `println()` и `main()`
- Знакомство с условными выражениями Kotlin, включая операторы `if/else` и булевы выражения.

Что вы узнаете

- Что такое null.
- Разница между nullable и non-nullable типами.
- Что такое безопасность null, ее важность и как в Kotlin достигается безопасность null.
- Как обращаться к методам и свойствам nullable переменных с помощью оператора безопасного вызова `?.` и оператора утверждения `!!` non-null.
- Как выполнять проверку нуля с помощью условий `if/else`.
- Как преобразовать переменную с нулевым типом к типу без нуля с помощью выражений `if/else`.
- Как с помощью выражения `if/else` или оператора `?: Elvis` предоставить значение по умолчанию, когда переменная с нулевым типом является нулевой.

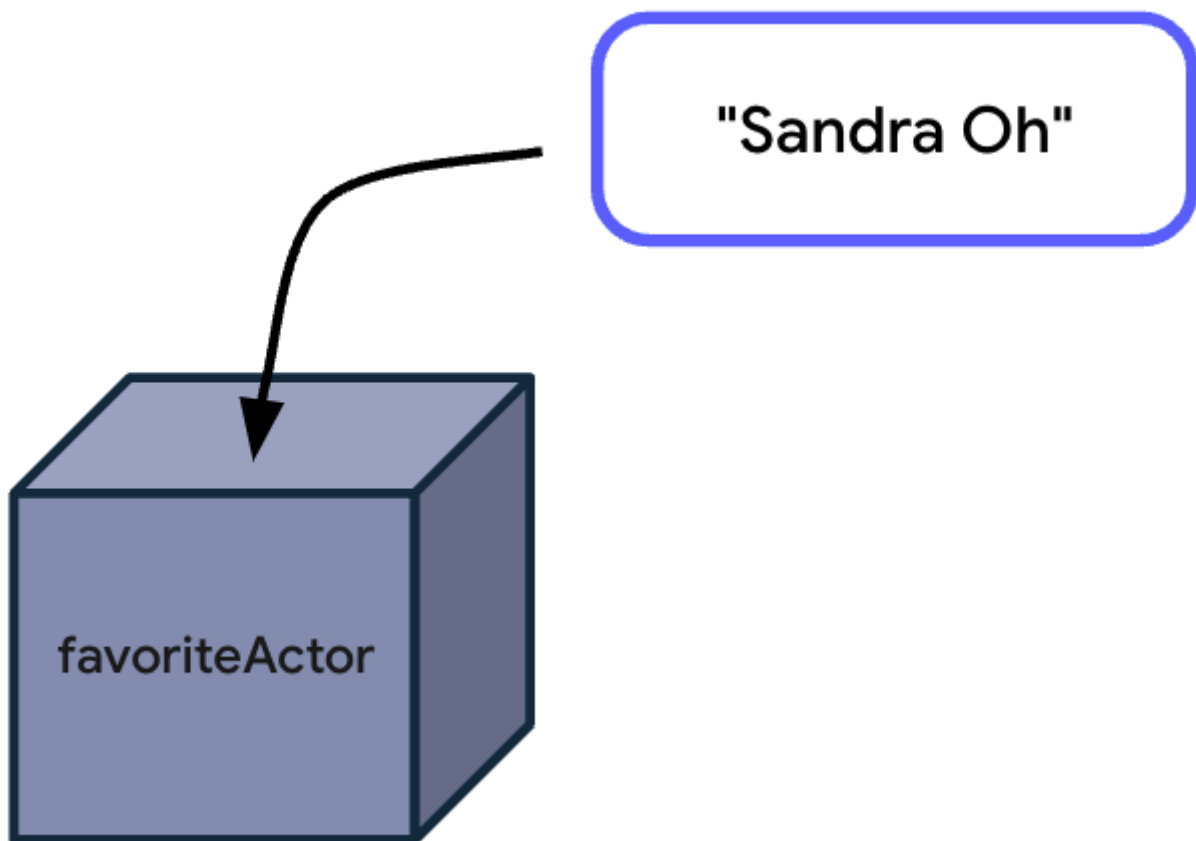
Что вам понадобится

- IntelliJ Idea

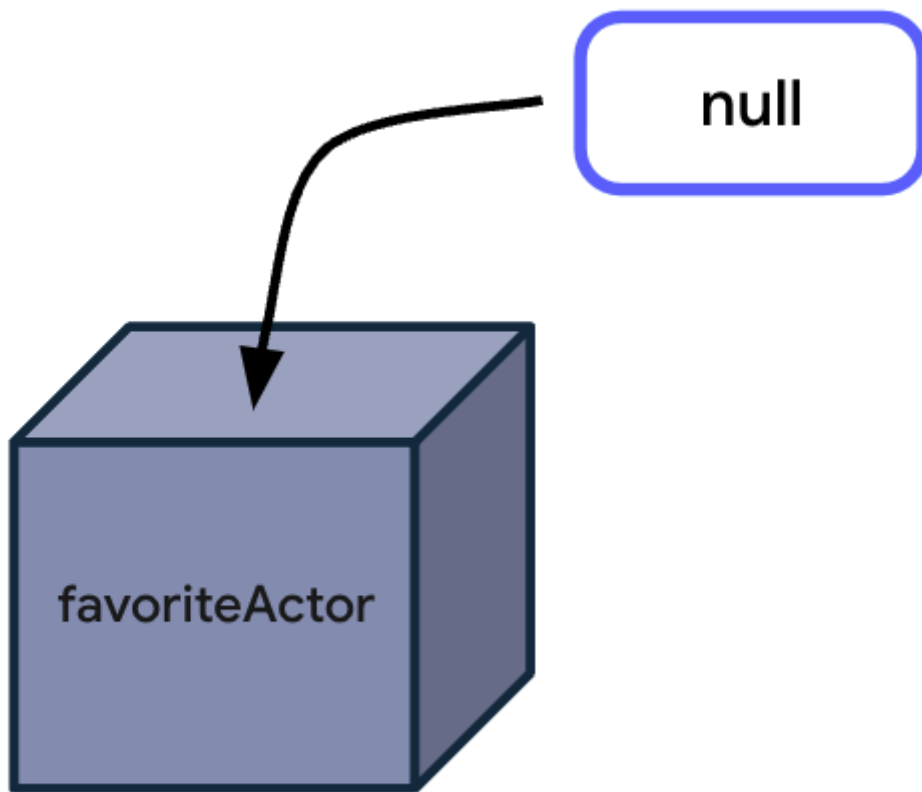
Применение переменных с null значением

Что такое null?

Когда вы объявляете переменную, вы должны сразу же присвоить ей значение. Например, когда вы объявляете переменную `favoriteActor`, вы можете сразу присвоить ей строковое значение «Sandra Oh».



Однако что делать, если у вас нет любимого актера? Возможно, вы захотите присвоить переменной значение «Nobody» или «None». Это не лучший подход, потому что ваша программа интерпретирует переменную `favoriteActor` как значение «Nobody» или «None», а не как отсутствие значения вообще. В Kotlin вы можете использовать `null`, чтобы указать на отсутствие значения, связанного с переменной.



Чтобы использовать null в коде, выполните следующие действия:

- замените содержимое тела функции main() на переменную favoriteActor, установленную в null:

```
fun main() {  
    val favoriteActor = null  
}
```

Выведите значение переменной favoriteActor с помощью функции println(), а затем запустите эту программу:

```
fun main() {  
    val favoriteActor = null  
    println(favoriteActor)  
}
```

Вывод выглядит так, как показано в этом фрагменте кода:

```
null
```

Переназначение переменных с помощью null

Ранее вы узнали, что можно переназначать переменные, определенные с помощью ключевого слова `var`, на другие значения одного типа. Например, вы можете переназначить переменную `name`, объявленную с одним именем, на другое имя, если новое имя имеет тип `String`.

```
var favoriteActor: String = "Sandra Oh"
favoriteActor = "Meryl Streep"
```

Бывают случаи, когда после объявления переменной вы хотите присвоить ей значение `null`. Например, после объявления любимого актера вы решаете, что не хотите раскрывать своего любимого актера вообще. В этом случае полезно присвоить переменной `favoriteActor` значение `null`.

Понимание ненулевых и нулевых переменных

Чтобы переназначить переменную `favoriteActor` на `null`, выполните следующие действия:

- замените ключевое слово `val` на ключевое слово `var`, а затем укажите, что переменная `favoriteActor` имеет тип `String`, и присвойте ей имя вашего любимого актера:

```
fun main() {
    var favoriteActor: String = "Sandra Oh"
    println(favoriteActor)
}
```

- удалите функцию `println()`:

```
fun main() {
    var favoriteActor: String = "Sandra Oh"
}
```

- переназначьте переменную `favoriteActor` на `null`, а затем запустите эту программу:

```
fun main() {
    var favoriteActor: String = "Sandra Oh"
    favoriteActor = null
}
```

Вы получите это сообщение об ошибке:

❗ Null can not be a value of a non-null type String

В Kotlin существует различие между nullable и non-nullable типами:

- Nullable типы - это переменные, которые могут содержать null.
- Ненулевые типы - это переменные, которые не могут содержать null.

Тип является nullable, только если вы явно разрешаете ему хранить null. Как говорится в сообщении об ошибке, тип данных String является ненулевым типом, поэтому вы не можете переназначить переменную на null.



Чтобы объявить нулевые переменные в Kotlin, нужно добавить оператор ? в конец типа. Например, тип `String?` может содержать либо строку, либо null, в то время как тип `String` может содержать только строку. Чтобы объявить переменную с нулем, необходимо явно добавить тип nullable. Без типа nullable компилятор Kotlin считает, что это не нулевой тип.

Измените тип переменной `favoriteActor` с типа данных `String` на тип данных `String?`

```
fun main() {  
    var favoriteActor: String? = "Sandra Oh"  
    favoriteActor = null  
}
```

Выведите переменную `favoriteActor` до и после переназначения нуля, а затем запустите эту программу:

```
fun main() {  
    var favoriteActor: String? = "Sandra Oh"  
    println(favoriteActor)  
  
    favoriteActor = null  
    println(favoriteActor)  
}
```

Вывод выглядит так, как показано в этом фрагменте кода:

```
Sandra Oh  
null
```

Изначально переменная `favoriteActor` содержала строку, а затем была переназначена на null.

Попробуйте Теперь, когда вы можете использовать тип nullable `String?`, можете ли вы инициализировать переменную значением `Int` и переназначить ее на null?

- запишите значение `Int` с возможностью обнуления. Удалите весь код в функции `main()`:

```
fun main() {  
  
}
```

- создайте числовую переменную с нулевым типом `Int`, а затем присвойте ей значение 10:

```
fun main() {  
    var number: Int? = 10  
}
```

- выведите числовую переменную, а затем запустите эту программу:

```
fun main() {  
    var number: Int? = 10  
    println(number)  
}
```

- выходные данные соответствуют ожиданиям:

10

- переназначьте переменную `number` на `null`, чтобы убедиться, что переменная может быть обнулена:

```
fun main() {  
    var number: Int? = 10  
    println(number)  
  
    number = null  
}
```

- добавьте еще один оператор `println(number)` в качестве заключительной строки программы и запустите ее:

```
fun main() {  
    var number: Int? = 10  
    println(number)  
  
    number = null  
    println(number)  
}
```

Выходные данные соответствуют ожиданиям:

```
10  
null
```

Примечание: Хотя для переменных, которые могут нести null, следует использовать переменные с нулями, для переменных, которые никогда не могут нести null, следует использовать переменные без нулей, поскольку доступ к переменным с нулями требует более сложной обработки. В следующем разделе вы узнаете о различных методах работы с нулевыми переменными.

Работа с нулевыми переменными

Ранее вы научились использовать оператор `.` для доступа к методам и свойствам ненулевых переменных. В этом разделе вы узнаете, как использовать его для доступа к методам и свойствам нулевых переменных.

Чтобы получить доступ к свойству ненулевой переменной `favoriteActor`, выполните следующие действия:

Удалите весь код в функции `main()`, а затем объявите переменную `favoriteActor` типа `String` и присвойте ей имя вашего любимого актера:

```
fun main() {  
    var favoriteActor: String = "Sandra Oh"  
}
```

- Выведите количество символов в значении переменной `favoriteActor` со свойством `length`, а затем запустите эту программу:

```
fun main() {  
    var favoriteActor: String = "Sandra Oh"  
    println(favoriteActor.length)  
}
```

Вывод соответствует ожиданиям: 9

В значении переменной `favoriteActor` девять символов, включая пробелы. Количество символов в имени вашего любимого актера может быть другим.

Доступ к свойству переменной с нулевым значением

Представьте, что вы хотите сделать переменную `favoriteActor` зануляемой, чтобы люди, у которых нет любимого актера, могли присвоить переменной значение `null`.

Чтобы получить доступ к свойству переменной `favoriteActor` с возможностью обнуления, выполните следующие действия:

Измените тип переменной `favoriteActor` на тип `nullable`, а затем запустите эту программу:

```
fun main() {  
    var favoriteActor: String? = "Sandra Oh"  
    println(favoriteActor.length)  
}
```

Вы получите это сообщение об ошибке:

❗ Only safe (?.) or non-null asserted (!!.) calls are allowed on a nullable receiver of type String?

Эта ошибка является ошибкой компиляции. Как уже говорилось, ошибка компиляции возникает, когда Kotlin не может скомпилировать код из-за синтаксической ошибки в вашем коде.

Kotlin намеренно применяет синтаксические правила, чтобы достичь `null safety`, что означает гарантию того, что не будет случайных вызовов потенциально нулевых переменных. Это не означает, что переменные не могут быть нулевыми. Это значит, что при обращении к члену переменной переменная не может быть нулевой.

Это очень важно, потому что если во время работы приложения попытаться получить доступ к члену переменной, который является нулевым - так называемая нулевая ссылка, - то приложение аварийно завершит работу, потому что нулевая переменная не содержит никакого свойства или метода. Этот тип сбоя известен как ошибка времени выполнения, когда ошибка происходит после компиляции и выполнения кода.

Благодаря нулевой безопасности Kotlin, такие ошибки времени выполнения предотвращаются, поскольку компилятор Kotlin принудительно проверяет нулевые типы. Под проверкой нуля понимается процесс проверки того, может ли переменная быть нулевой, до того, как к ней будет получен доступ и она будет обработана как ненулевой тип. Если вы хотите использовать значение с нулевым значением в качестве ненулевого типа, вам необходимо явно выполнить проверку на ноль.

В этом примере код не работает во время компиляции, потому что прямое обращение к свойству `length` для переменной `favoriteActor` недопустимо, так как существует вероятность того, что переменная равна `null`.

Далее вы изучите различные техники и операторы для работы с нулевыми типами.

Использование оператора `?.` safe call

Вы можете использовать оператор `?.` для доступа к методам или свойствам переменных с нулевым типом.

nullable variable ? . method/property

Чтобы использовать оператор `?.` для доступа к методу или свойству, добавьте символ `?` после имени переменной и обратитесь к методу или свойству с помощью нотации `.`

Оператор безопасного вызова `?.` обеспечивает более безопасный доступ к переменным с нулевым значением, поскольку компилятор Kotlin пресекает любые попытки доступа к члену со ссылками на `null` и возвращает `null` для члена, к которому осуществляется доступ.

Чтобы безопасно получить доступ к свойству нулевой переменной `favoriteActor`, выполните следующие действия:

- В операторе `println()` замените оператор `.` на оператор безопасного вызова `?.`:

```
fun main() {  
    var favoriteActor: String? = "Sandra Oh"  
    println(favoriteActor?.length)  
}
```

- Запустите эту программу и убедитесь, что результат соответствует ожиданиям: 9

Количество символов в имени вашего любимого актера может отличаться.

- Переназначьте переменную `favoriteActor` на `null`, а затем запустите эту программу:

```
fun main() {  
    var favoriteActor: String? = null  
    println(favoriteActor?.length)  
}
```

Вы видите следующее сообщение: `null`

Обратите внимание, что программа не аварийно завершается, несмотря на попытку получить доступ к свойству `length` переменной `null`. Выражение безопасного вызова просто возвращает `null`.

Примечание: Вы также можете использовать операторы безопасного вызова `?.` для доступа к методу или свойству переменной, не являющейся `null`. Хотя компилятор Kotlin не выдаст никакой ошибки, в этом нет необходимости, поскольку доступ к методам или свойствам для ненулевых переменных всегда безопасен.

Использование оператора утверждения `!!`

Вы также можете использовать оператор утверждения `!!` для доступа к методам или свойствам переменных с нулевым значением.

nullable variable**!!****method/property**

После переменной nullable нужно добавить оператор утверждения **!!**, за которым следует оператор **.** и затем метод или свойство без пробелов.

Как следует из названия, если вы используете утверждение **!!**, это означает, что вы утверждаете, что значение переменной не является нулевым, независимо от того, является она таковой или нет.

В отличие от операторов безопасного вызова **?.**, использование оператора утверждения **!!** not-null может привести к ошибке `NullPointerException`, если nullable переменная действительно является null. Таким образом, это следует делать только в том случае, если переменная всегда является ненулевой или если установлена надлежащая обработка исключений. Если исключения не обрабатываются, они приводят к ошибкам во время выполнения. Об обработке исключений вы узнаете в последующих разделах этого курса.

Чтобы получить доступ к свойству переменной `favoriteActor` с помощью оператора утверждения **!!** not-null, выполните следующие действия:

- Переназначьте переменную `favoriteActor` на имя вашего любимого актера, а затем замените оператор **?.** safe-call на оператор утверждения **!!** not-null в операторе `println()`:

```
fun main() {  
    var favoriteActor: String? = "Sandra Oh"  
    println(favoriteActor!!.length)  
}
```

- Запустите эту программу и убедитесь, что результат соответствует ожиданиям: 9

Количество символов в имени вашего любимого актера может отличаться.

Переназначьте переменную `favoriteActor` на null, а затем запустите эту программу:

```
fun main() {  
    var favoriteActor: String? = null  
    println(favoriteActor!!.length)  
}
```

Вы получите ошибку `NullPointerException`:

```
Exception in thread "main" java.lang.NullPointerException  
    at FileKt.main (File.kt:4)  
    at FileKt.main (File.kt:-1)  
    at jdk.internal.reflect.NativeMethodAccessorImpl.invoke0 (: -2)
```

Эта ошибка Kotlin показывает, что ваша программа разрушилась во время выполнения. Поэтому не рекомендуется использовать оператор утверждения `!! not-null`, если вы не уверены, что переменная не является `null`.

Примечание: В других языках программирования, не содержащих атрибута `null-safety`, ошибка `NullPointerException` является частой причиной сбоев приложений. Таким образом, Kotlin устраняет огромную причину сбоев программ, поскольку включает в язык атрибут `null safety`.

Используйте условные элементы `if/else`

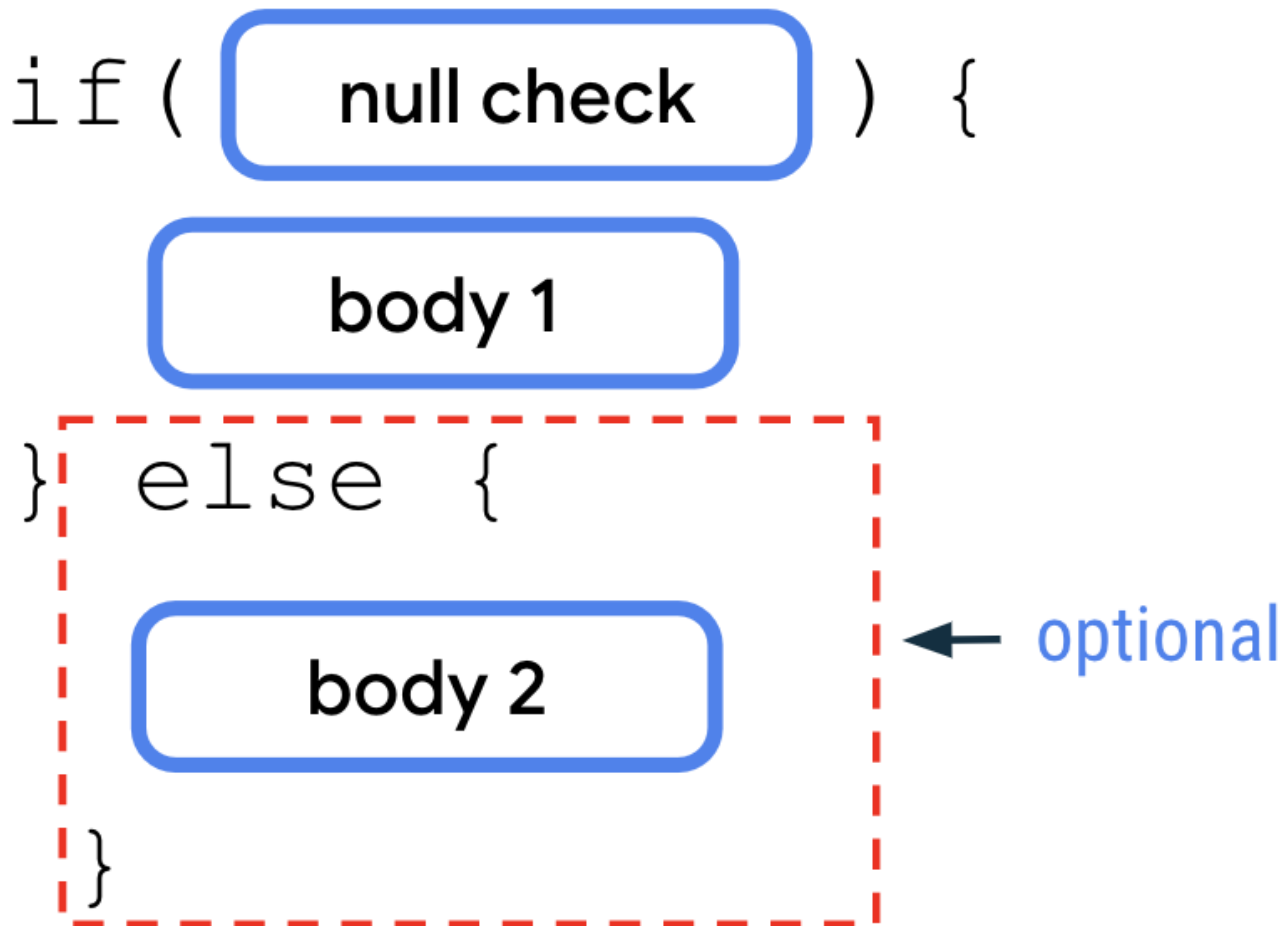
Вы можете использовать ветвь `if` в условных операторах `if/else` для проверки нуля.

nullable variable `!= null`

Для проверки нуля можно проверить, что переменная с нулевым значением не равна нулю, с помощью оператора сравнения `!=`.

Операторы `if/else`

Оператор `if/else` можно использовать вместе с проверкой нуля следующим образом:



Проверка нуля полезна в сочетании с операторами `if/else`:

Проверка на ноль выражения `nullableVariable != null` используется в качестве условия `if`.

В теле 1 внутри ветви `if` предполагается, что переменная не является `null`. Поэтому в этом теле можно свободно обращаться к методам или свойствам переменной, как если бы это была ненулевая переменная, без использования оператора безопасного вызова `?.` или оператора утверждения `!! not-null`.

В теле 2 внутри ветви `else` предполагается, что переменная является нулевой. Поэтому в это тело можно добавить операторы, которые должны выполняться, когда переменная равна `null`. Ветвь `else` является необязательной. Вы можете использовать только условие `if` для запуска проверки на `null`, не предусматривая действий по умолчанию при неудачной проверке на `null`. Проверку на ноль удобнее использовать с условием `if`, когда есть несколько строк кода, использующих переменную с нулевым значением. В отличие от этого, оператор безопасного вызова `?.` удобнее использовать для единственной ссылки на нулевую переменную.

Чтобы написать оператор `if/else` с проверкой нуля для переменной `favoriteActor`, выполните следующие действия:

- Снова присвойте переменной `favoriteActor` имя любимого актера, а затем удалите оператор `println()`:

```
fun main() {  
    var favoriteActor: String? = "Sandra Oh"  
}
```

- Добавьте ветку if с условием favoriteActor != null:

```
fun main() {  
    var favoriteActor: String? = "Sandra Oh"  
  
    if (favoriteActor != null) {  
  
    }  
}
```

- В тело ветви if добавьте оператор println, который принимает строку «Количество символов в имени вашего любимого актера равно \${favoriteActor.length}.», а затем запустите эту программу:

```
fun main() {  
    var favoriteActor: String? = "Sandra Oh"  
  
    if (favoriteActor != null) {  
        println("The number of characters in your favorite actor's name is  
${favoriteActor.length}.")  
    }  
}
```

Выходные данные соответствуют ожиданиям.

The number of characters in your favorite actor's name is 9.

Количество символов в имени вашего любимого актера может отличаться.

Обратите внимание, что вы можете получить прямой доступ к методу длины имени с помощью оператора ., потому что вы обращаетесь к методу длины внутри ветви if после проверки на null. Таким образом, компилятор Kotlin знает, что вероятность того, что переменная favoriteActor равна null, исключена, поэтому компилятор разрешает прямой доступ к свойству.

Необязательно: Добавьте ветку else для обработки ситуации, когда имя актера равно null:

```
fun main() {  
    var favoriteActor: String? = "Sandra Oh"  
  
    if (favoriteActor != null) {  
        println("The number of characters in your favorite actor's name is  
${favoriteActor.length}.")  
    } else {
```

```
    }  
}
```

В теле ветви `else` добавьте оператор `println`, который принимает строку «Вы не ввели имя.»:

```
fun main() {  
    var favoriteActor: String? = "Sandra Oh"  
  
    if (favoriteActor != null) {  
        println("The number of characters in your favorite actor's name is  
${favoriteActor.length}.")  
    } else {  
        println("You didn't input a name.")  
    }  
}
```

Присвойте переменной `favoriteActor` значение `null`, а затем запустите эту программу:

```
fun main() {  
    var favoriteActor: String? = null  
  
    if(favoriteActor != null) {  
        println("The number of characters in your favorite actor's name is  
${favoriteActor.length}.")  
    } else {  
        println("You didn't input a name.")  
    }  
}
```

Вывод соответствует ожиданиям:

Вы не ввели имя.

Вы также можете объединить проверку на ноль с выражением `if/else`, чтобы преобразовать переменную с нулевым значением в переменную без нулевого значения.

```

val name : non-null type = if ( null check ) {
    body 1
} else {
    body 2
}

```

Для присвоения выражения if/else типу, не являющемуся нулевым:

В качестве условия if используется проверка nullableVariable != null. Тело 1 внутри ветви if предполагает, что переменная не является null. Поэтому в этом теле можно получить доступ к методам или свойствам переменной, как если бы это была ненулевая переменная, без оператора безопасного вызова ?. или оператора утверждения !! not-null.

В теле 2 внутри ветви else предполагается, что переменная является нулевой. Поэтому в это тело можно добавить операторы, которые должны выполняться, когда переменная равна null.

В последней строке тела 1 и 2 нужно использовать выражение или значение, которое приводит к типу not-nullable, чтобы оно присваивалось переменной not-nullable, когда проверка на null пройдет или не пройдет соответственно. Чтобы использовать выражение if/else для переписывания программы таким образом, чтобы в ней использовался только один оператор println, выполните следующие действия:

- Присвойте переменной favoriteActor имя вашего любимого актера:

```

fun main() {
    var favoriteActor: String? = "Sandra Oh"

    if (favoriteActor != null) {
        println("The number of characters in your favorite actor's name is
        ${favoriteActor.length}.")
    } else {
        println("You didn't input a name.")
    }
}

```

- Создайте переменную lengthOfName и присвойте ее выражению if/else:

```

fun main() {
    var favoriteActor: String? = "Sandra Oh"

    val lengthOfName = if (favoriteActor != null) {
        println("The number of characters in your favorite actor's name is
        ${favoriteActor.length}.")
    }
}

```

```
    } else {  
        println("You didn't input a name.")  
    }  
}
```

- Удалите оба оператора println() из ветвей if и else:

```
fun main() {  
    var favoriteActor: String? = "Sandra Oh"  
  
    val lengthOfName = if (favoriteActor != null) {  
  
    } else {  
  
    }  
}
```

- В теле ветви if добавьте выражение favoriteActor.length:

```
fun main() {  
    var favoriteActor: String? = "Sandra Oh"  
  
    val lengthOfName = if (favoriteActor != null) {  
        favoriteActor.length  
    } else {  
  
    }  
}
```

Доступ к свойству length переменной favoriteActor осуществляется непосредственно с помощью оператора .

- В теле ветви else добавьте значение 0:

```
fun main() {  
    var favoriteActor: String? = "Sandra Oh"  
  
    val lengthOfName = if (favoriteActor != null) {  
        favoriteActor.length  
    } else {  
        0  
    }  
}
```

Значение 0 служит значением по умолчанию, когда имя равно null.

В конце функции `main()` добавьте оператор `println`, который принимает строку «Количество символов в имени вашего любимого актера равно `$lengthOfName`», а затем запустите эту программу:

```
fun main() {
    var favoriteActor: String? = "Sandra Oh"

    val lengthOfName = if (favoriteActor != null) {
        favoriteActor.length
    } else {
        0
    }

    println("The number of characters in your favorite actor's name is
    $lengthOfName.")
}
```

Выходные данные соответствуют ожиданиям:

Количество символов в имени вашего любимого актера равно 9.

Количество символов в имени, которое вы использовали, может отличаться.

Примечание: Для проверки нуля можно также использовать оператор сравнения `==` вместо оператора `!=`. При этом обратите внимание на то, что два тела поменялись местами. В теле ветви `if` предполагается, что переменная равна `null`, а в теле ветви `else` - что переменная не равна `null`.

Использование оператора `?:` Elvis

Оператор `?:` Elvis - это оператор, который можно использовать вместе с оператором `?.` `safe-call`. С помощью оператора `?:` Elvis вы можете добавить значение по умолчанию, когда оператор `?.` `safe-call` возвращает `null`. Это похоже на выражение `if/else`, но в более идиоматическом виде.

Если переменная не равна `null`, выполняется выражение перед оператором `?:` Elvis. Если переменная равна `null`, выполняется выражение после оператора `?:` Elvis.

`val` name `=` nullable variable `?.` method/property `?:` default value

Чтобы модифицировать предыдущую программу для использования оператора `?:` Elvis, выполните следующие действия:

- Удалите условие `if/else`, затем установите переменную `lengthOfName` в нулевую переменную `favoriteActor` и используйте оператор `?.` `safe-call` для вызова ее свойства `length`:

```
fun main() {
    var favoriteActor: String? = "Sandra Oh"
```

```
val lengthOfName = favoriteActor?.length

println("The number of characters in your favorite actor's name is
$lengthOfName.")
}
```

- После свойства `length` добавьте оператор `?:` Elvis, за которым следует значение `0`, а затем запустите эту программу:

```
fun main() {
    var favoriteActor: String? = "Sandra Oh"

    val lengthOfName = favoriteActor?.length ?: 0

    println("The number of characters in your favorite actor's name is
$lengthOfName.")
}
```

Выходной сигнал такой же, как и в предыдущем случае:

```
The number of characters in your favorite actor's name is 9.
```

Примечание: Оператор `?:` Elvis назван в честь рок-звезды Элвиса Пресли, потому что при боковом просмотре он напоминает смайлик с его челкой.

Заключение

- Вы узнали о нулевости и о том, как использовать различные операторы для управления ею.

Резюме

- Переменная может быть установлена в `null`, чтобы указать, что она не имеет значения.
- Не нулевым переменным нельзя присвоить значение `null`.
- Нулевым переменным можно присвоить значение `null`.
- Для доступа к методам или свойствам nullable переменных необходимо использовать операторы `?.` safe-call или `!!` not-null assertion.
- Для доступа к нулевым переменным в ненулевых контекстах можно использовать операторы `if/else` с проверкой на ноль.
- С помощью выражений `if/else` можно преобразовать переменную с нулевым типом к типу без нуля.
- С помощью выражений `if/else` или оператора `?:` Elvis можно задать значение по умолчанию для случая, когда переменная с нулевым типом является нулевой.